

Linguaggi Formali e Compilatori

Argomento 02:

Espressioni e Linguaggi Regolari

Prof. Giuseppe Psaila

Laurea Magistrale in Ingegneria Informatica
Università di Bergamo

- Le **ESPRESSIONI REGOLARI** sono definite sfruttando gli operatori di **Unione**, **Concatenamento** e **Stella**.
- Dispongono di un meccanismo efficiente di riconoscimento: gli **AUTOMI A STATI FINITI**.

Definizione: Linguaggio Regolare

- Un linguaggio di alfabeto $\Sigma = \{a_1, a_2, \dots, a_n\}$ è detto **REGOLARE** se può essere espresso mediante le operazioni di **Concatenamento**, **Unione** e **Stella** applicate un numero **Finito di Volte** ai linguaggi unitari $\{a_1\}$, $\{a_2\}$, $\dots$, $\{a_n\}$ e al linguaggio vuoto $\emptyset$.

Definizione: Espressione Regolare

- Un'**ESPRESSIONE REGOLARE** (abbreviata come e.r.) r

è una stringa costruita con i caratteri dell'alfabeto Σ ,
con i meta-simboli $\{\bullet, \cup, *, \emptyset\}$
e con le parentesi tonde.

Si suppone che i meta-simboli non facciano parte
dell'alfabeto Σ .

Regole di Costruzione

- ❶ $r = \emptyset$
- ❷ $r = a$ dove $a \in \Sigma$
- ❸ $r = (s \cup t)$ dove s e t sono e.r.
- ❹ $r = (s \bullet t)$ dove s e t sono e.r.
o anche $r = (st)$
- ❺ $r = (s)^*$ dove s è un'e.r.

Anche il simbolo ϵ può essere usato in una e.r., poiché vale l'identità $\epsilon = \emptyset^*$.

Precedenza degli operatori: $*$, $\bullet$, $\cup$.

Espressioni Regolari e Linguaggi

- Una espressione regolare **Definisce** o **Genera** un linguaggio.

Notazioni Comode

- $e^k = ee \dots e \quad k \geq 0$ volte
- $e^+ = ee^*$
- $[e]_k^h = e^k \cup e^{k+1} \cup \dots \cup e^h$
con $0 \leq k \leq h$
Ripetizione da k a h volte
- $[e] = [e]_0^1 = \epsilon \cup e$
Opzionalità

Esempio

Dato $\Sigma = \{0, 1\}$

- Struttura dei numeri binari di lunghezza arbitraria
$$e = (0 \cup 1)(0 \cup 1)^* = (0 \cup 1)^+$$
- Struttura dei numeri binari di lunghezza fissa k
$$e = (0 \cup 1)^k$$

Esercizio 2.1

Dato $\Sigma = \{a, n\}$ (a indica una generica lettera alfabetica, n un generico carattere numerico),

definire il **Linguaggio degli Identificatori**, dove un identificatore inizia con una lettera alfabetica e prosegue con una sequenza arbitraria di lettere e numeri.

- Di lunghezza non limitata

$e =$

- Di lunghezza massima k

$e =$

Sotto-Espressione (S.E.)

Una Sotto-Espressione (S.E.) di un'espressione regolare è così definita.

- e_k è una S.E. di $(e_1 \cup e_2 \cup \dots \cup e_k \cup \dots \cup e_j)$
- e_k è una S.E. di $(e_1 \bullet \dots \bullet e_k \bullet \dots \bullet e_j)$
- e è una S.E. di e^* , e^+ e di e^k
- ϵ è una S.E. di ogni espressione regolare

Per proseguire, definiamo il concetto di **Relazione di Implicazione**.

Relazione di Implicazione $\Rightarrow$

$e' \Rightarrow e''$ (e' implica e'') se le due espressioni si possono fattorizzare come

$$e' = \alpha\beta\gamma \quad \text{e} \quad e'' = \alpha\delta\gamma$$

dove α , β e γ sono S.E. di e'

e tra β e δ vale una delle seguenti relazioni.

β	δ
$(e_1 \cup \dots \cup e_k \cup \dots \cup e_h)$	e_k con $1 \leq k \leq h$
e^*	$ee \dots e$ $k \geq 0$ volte
e^+	$ee \dots e$ $k \geq 1$ volte
e^n	$ee \dots e$ n volte

- L'implicazione può essere **Applicata Transitivamente**

$e_0 \xRightarrow{n} e_n$ si dice che e_0 implica e_n in un numero n di passi, cioè

$$e_0 \Rightarrow e_1 \Rightarrow \cdots \Rightarrow e_{n-1} \Rightarrow e_n$$

Se $n > 1$, l'Implicazione è detta **NON IMMEDIATA**
o **TRANSITIVA**.

- Le scritture $e_0 \xRightarrow{*} e_n$ e $e_0 \xRightarrow{+} e_n$ indicano che e_n è ottenuta da e_0 in un numero di passi $n \geq 0$ (risp. $n > 0$).
Se $n = 0$, risulta $e_n = e_0$.

Linguaggio Generato da un'Espressione Regolare r

$$L(r) = \{x \in \Sigma^* \mid r \xRightarrow{*} x\}$$

con x **Priva di Meta-Simboli**.

Un linguaggio è detto **REGOLARE** se è **Generato da un'Espressione Regolare**.

Due espressioni regolari sono **EQUIVALENTI** se **Generano lo Stesso Linguaggio**.

Implicazione Sinistra

Un'Implicazione è detta **SINISTRA**

se, data $e' \Rightarrow e''$, e' e e'' si fattorizzano come

$$e' = \alpha\beta\gamma \quad \text{e} \quad e'' = \alpha\delta\gamma$$

dove α è il **pù lungo prefisso** comune ad e' e e''
privo di meta-simboli.

Altri Operatori

- **Complemento**

$\neg e$

$$L(\neg e) = \Sigma^* - L(e)$$

- **Intersezione**

$e_1 \cap e_2$

$$L(e_1 \cap e_2) = L(e_1) \cap L(e_2)$$

Esempio

- $(a^* \cup b^+) \Rightarrow a^*$
 $(a^* \cup b^+) \Rightarrow b^+$
- $(a^* \cup b^+) \Rightarrow a^* \Rightarrow \epsilon$, quindi $(a^* \cup b^+) \xRightarrow{2} \epsilon$
generalizzabile in $(a^* \cup b^+) \xRightarrow{+} \epsilon$
- $(a^* \cup b^+) \Rightarrow b^+ \Rightarrow bb$, quindi $(a^* \cup b^+) \xRightarrow{2} bb$
generalizzabile come $(a^* \cup b^+) \xRightarrow{+} bb$

Esempio

- $(ab)^* \Rightarrow abab$
- $(ab \cup c) \Rightarrow ab$
- $a(ba \cup c)^*d \Rightarrow ad$
- $a(ba \cup c)^*d \Rightarrow a(ba \cup c)(ba \cup c)d$
- $a^*(b \cup c \cup d)f^+ \Rightarrow aa(b \cup c \cup d)f^+$
- $a^*(b \cup c \cup d)f^+ \Rightarrow a^*cf^+ \Rightarrow aaacf^+ \Rightarrow aaacf$

Sia Θ un operatore che, applicato ad uno o due linguaggi, ne produce un altro.

Una famiglia di linguaggi si dice **CHIUSA** rispetto ad un operatore Θ se il linguaggio risultante appartiene alla stessa famiglia dei linguaggi cui Θ viene applicato.

- La famiglia dei **Linguaggi Regolari** è **Chiusa** rispetto agli operatori **Concatenamento**, **Unione** e **Stella**.

Proprietà

La famiglia dei Linguaggi Regolari è la **PIÙ PICCOLA FAMIGLIA** di linguaggi che:

- 1 contiene **Tutti i Linguaggi Finiti**;
- 2 è **Chiusa** rispetto a **Concatenamento**, **Unione** e **Stella**.

Inoltre, la famiglia dei Linguaggi Regolari è chiusa rispetto a **Complemento**, **Intersezione** e **Ripetizione**.

Automa a Stati Finiti Deterministico

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : Insieme degli Stati
- Σ : alfabeto di ingresso
- δ : Funzione di Transizione
 $\delta : Q \times \Sigma \rightarrow Q$
- $q_0 \in Q$ Stato Iniziale
- $F \subseteq Q$: Stati Finali

Funzione di Transizione Transitiva

$$\delta^*(q, ya) = \delta(\delta^*(q, y), a)$$

$$\text{con } \delta^*(q, \epsilon) = q.$$

Una stringa $x \in L(r)$ (dove r è un'espressione regolare riconosciuta dall'automa) se

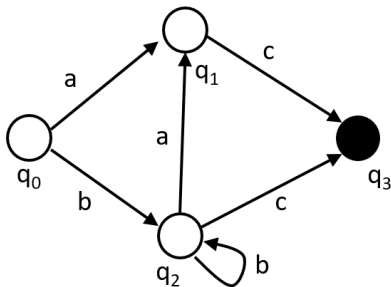
$$\delta^*(q_0, x) = q \text{ con } q \in F.$$

Automi a Stati Finiti

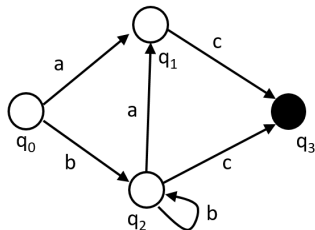
$$\Sigma = \{a, b, c\}$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$F = \{q_3\}$$

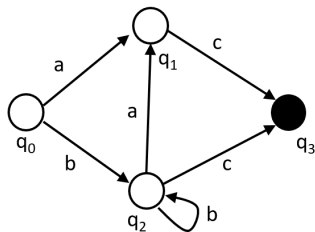


Esecuzione Corretta



Input	Stato	Decisione
<i>bbac</i>	<i>q₀</i>	Leggi
<i>bac</i>	<i>q₂</i>	Leggi
<i>ac</i>	<i>q₂</i>	Leggi
<i>c</i>	<i>q₁</i>	Leggi
	<i>q₃</i>	OK

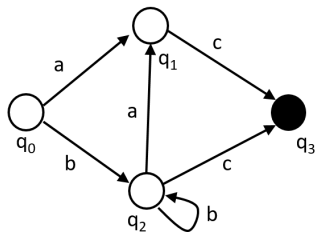
Esecuzione Scorretta



Input	Stato	Decisione
<i>acc</i>	q_0	Leggi
<i>cc</i>	q_1	Leggi
<i>c</i>	q_3	Errore

La stringa in Input **non è vuota**

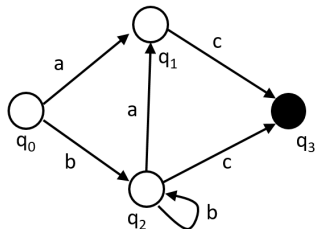
Esecuzione Scorretta



Input	Stato	Decisione
<i>abc</i>	<i>q₀</i>	Leggi
<i>bc</i>	<i>q₁</i>	Errore

Non esistono mosse uscenti da q_1 con b

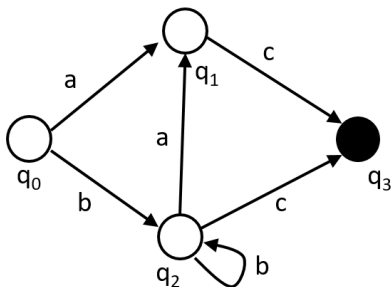
Esecuzione Scorretta



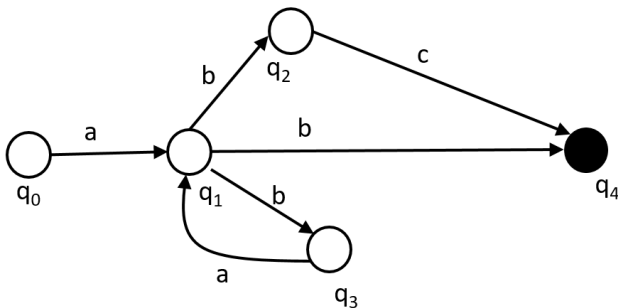
Input	Stato	Decisione
<i>ba</i>	q_0	Leggi
<i>a</i>	q_2	Leggi
	q_1	Errore

La stringa in Input è finita, ma l'Automa **non è in uno stato finale**

Esercizio 2.2: Espressione Regolare?



Automa Non Deterministico



Automa a Stati Finiti Non Deterministico senza ϵ -Mosse

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : Insieme degli Stati
- Σ : alfabeto di ingresso
- δ : Funzione di Transizione Non-Deterministica
 $\delta : Q \times \Sigma \rightarrow (2^Q - \{\emptyset\})$
- $q_0 \in Q$ Stato Iniziale
- $F \subseteq Q$: Stati Finali

Insieme delle Parti

Dato un insieme Q , l'insieme delle sue parti 2^Q è

l'Insieme di Tutti i Suoi Sottoinsiemi

Si noti che $|2^Q| = 2^{|Q|}$

Esempio

Se $Q = \{a, b, c\}$

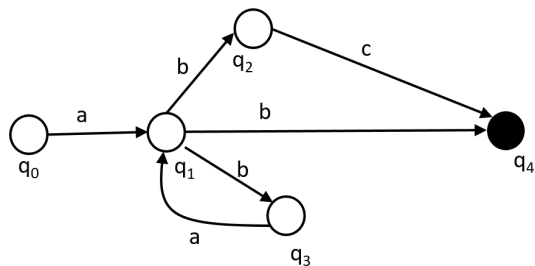
$2^Q = \{\{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}, \emptyset\}$

Funzione di Transizione Transitiva Non Deterministica senza ϵ -Mosse

$\delta^*(q, ya) = \{p \mid \exists r \in \delta^*(q, y) \text{ AND } p \in \delta(r, a)\}$
con $\delta^*(q, \epsilon) = \{q\}$.

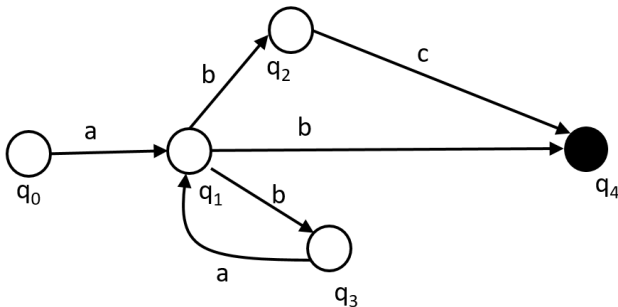
Una stringa $x \in L(r)$ (dove r è un'espressione regolare riconosciuta dall'automa) se
 $\exists q \in F$ tale che $q \in \delta^*(q_0, x)$.

Automa Non Deterministico



Input	Stato	Decisione
<i>abab</i>	q_0	Leggi
<i>bab</i>	q_1	Scegli q_2
<i>ab</i>	q_2	BackTrack
<i>bab</i>	q_1	Scegli q_3
<i>ab</i>	q_3	Leggi
<i>b</i>	q_1	Scegli q_4
	q_4	OK

Esercizio 2.3: Espressione Regolare?



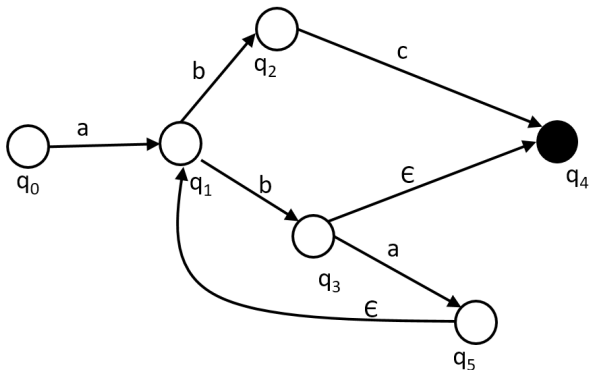
Epsilon-Mosse

Una **epsilin-mossa** è una transizione diretta da uno stato all'altro etichettata con ϵ .

Viene detta **MOSSA SPONTANEA**, perchè la transizione avviene consumando il carattere ϵ dal dispositivo di ingresso, cioè **senza consumare alcun carattere (spontaneamente)**.

In opposizione, le altre mosse vengono dette **NON SPONTANEE**.

Automa Non Deterministico con Epsilon Mosse



Automa a Stati Finiti Non Deterministico con ϵ -Mosse

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : Insieme degli Stati
- Σ : alfabeto di ingresso
- δ : Funzione di Transizione Non-Deterministica
 $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow (2^Q - \{\emptyset\})$
- $q_0 \in Q$ Stato Iniziale
- $F \subseteq Q$: Stati Finali

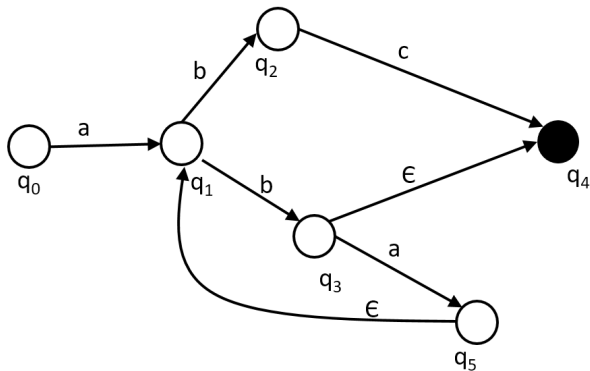
Funzione di Transizione Transitiva Non Deterministica con ϵ -Mosse

$$\delta^*(q, ya) = \{p \mid \exists r \in \delta^*(q, y) \text{ AND } p \in \delta(r, a)\}$$

con $\delta^*(q, \epsilon) = \{q\}$ se $\delta(q, \epsilon)$ non è definita,
dove $a \in \Sigma \cup \{\epsilon\}$.

Una stringa $x \in L(r)$ (dove r è un'espressione regolare riconosciuta dall'automa) se
 $\exists q \in F$ tale che $q \in \delta^*(q_0, x)$.

Automa Non Deterministico con Epsilon Mosse



$$r_1 = a(bc \cup (b(ab)^*[c]))$$

Esercizio 2.4: Trasformiamo l'Espressione Regolare

$$r_1 = a(bc \cup (b(ab)^*[c]))$$

$$r_2 =$$

Esercizio 2.5

Automa Non Deterministico con Epsilon Mosse

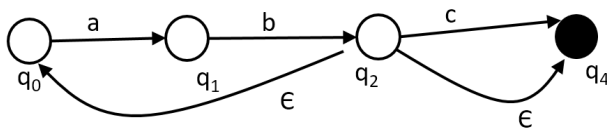
$$r_4 = ab(ab)^*[c]$$

?

Automa Non Deterministico con Epsilon Mosse

$r_4 = ab(ab)^*[c]$ è equivalente a

$r_5 = (ab)^+[c]$

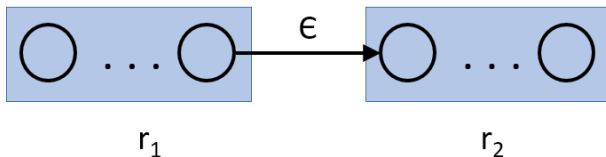


Utilità delle EPsilon-Mosse

- Le ϵ -mosse servono per costruire automi
- a partire da espressioni regolari di qualsiasi complessità
- Perché consentono di comporre liberamente gli automi derivati dalle S.E.

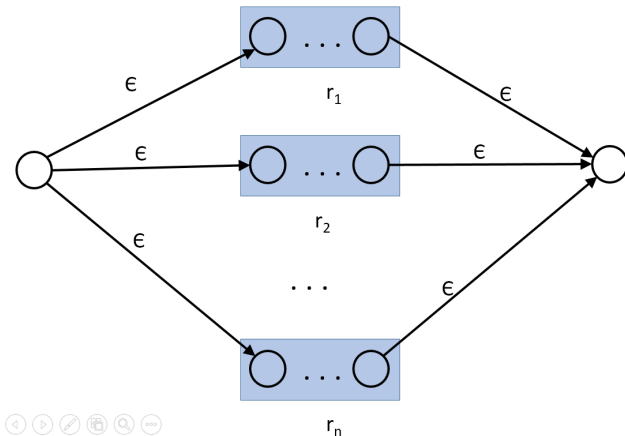
Composizione: Concatenamento

$$r = r_1 \bullet r_2$$



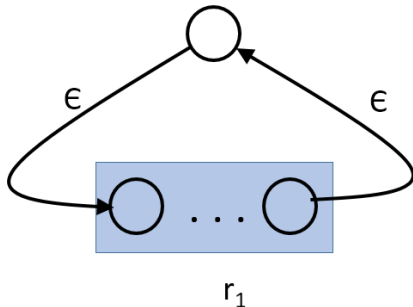
Composizione: Alternativa

$$r = (r_1 \cup r_2 \cup \dots \cup r_n)$$



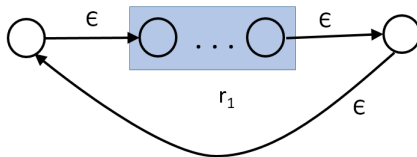
Composizione: Stella

$$r = (r_1)^*$$



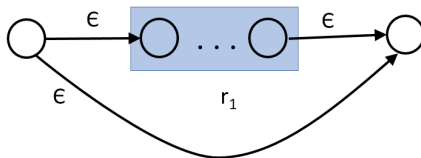
Composizione: Croce

$$r = (r_1)^+$$



Composizione: Opzionalità

$$r = [r_1]$$



Linguaggio dei Numeri in Base 10

$$\Sigma = \{d, '+', '-', ', '\}$$

$$i = ['+' \cup '-'] d^+$$

$$v = ['+' \cup '-'] (d^+ , ' d^+)$$

$$r = (i \cup v)$$

Semplifichiamo l'espressione

$$r_s =$$

Linguaggio dei Numeri in Base 10

$$\Sigma = \{d, '+', '-', ', ', '\}$$

$$i = ['+' \cup '-'] d^+$$

$$v = ['+' \cup '-'] (d^+ , , d^+)$$

$$r = (i \cup v)$$

Semplifichiamo l'espressione

$$r_1 = (['+' \cup '-'] d^+ \cup ['+' \cup '-'] (d^+ , , d^+))$$

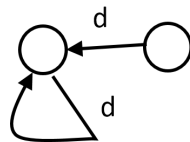
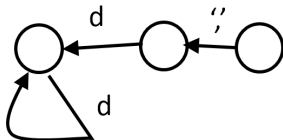
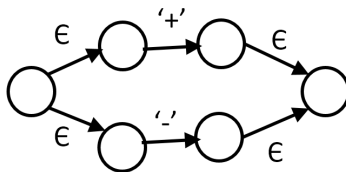
$$r_2 = ['+' \cup '-'] (d^+ \cup (d^+ , , d^+))$$

$$r_3 = ['+' \cup '-'] d^+ (\epsilon \cup , , d^+)$$

$$r_s = ['+' \cup '-'] d^+ [, , d^+]$$

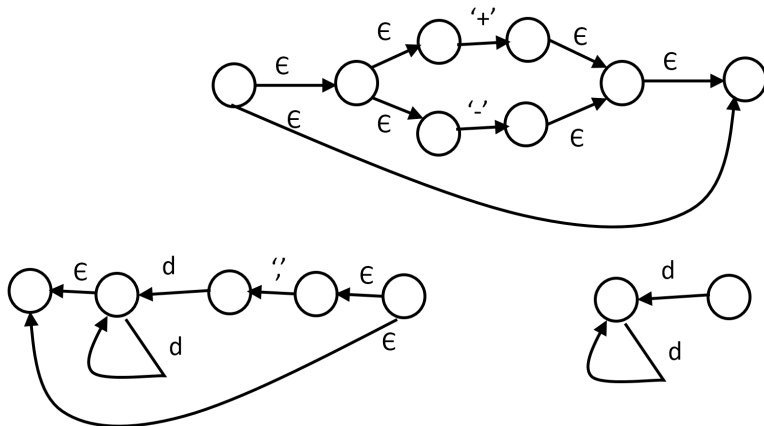
Componiamo l'Automa per il Linguaggio dei Numeri in Base 10

$$r_s = ['+' \cup '-'] d^+ [',' d^+]$$



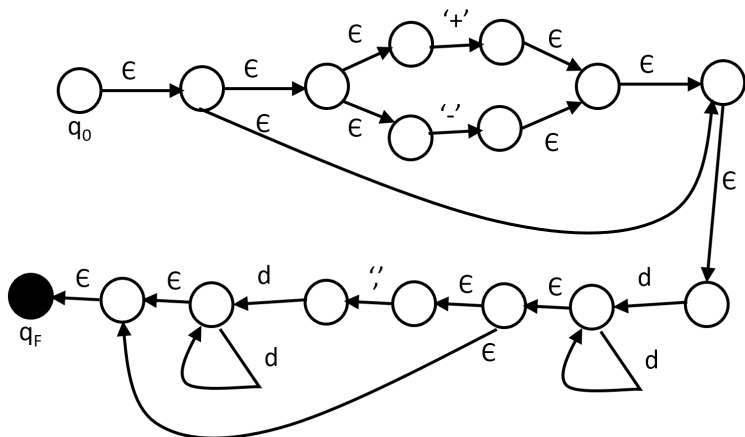
Componiamo l'Automa per il Linguaggio dei Numeri in Base 10

$$r_s = ['+' \cup '-'] d^+ [, d^+]$$



Componiamo l'Automa per il Linguaggio dei Numeri in Base 10

$$r_s = ['+' \cup '-'] d^+ [', ' d^+]$$



Complemento

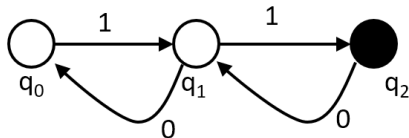
Dato un automa finito **Deterministico** M che riconosce il linguaggio L , derivare l'automa deterministico $\overline{M}$ che riconosce il linguaggio $\neg L$.

Esiste un algoritmo che deriva l'automa a stati finiti deterministico $\overline{M}$ dall'automa a stati finiti deterministico M .

Algoritmo Complemento

- Sia $M(Q, \Sigma, \delta, q_0, F)$ l'automa deterministico per L .
- Sia $\overline{M} = (\overline{Q}, \Sigma, \overline{\delta}, q_0, \overline{F})$ la'utoma per $\neg L$ da calcolare.
- **(1)** Aggiungere agli stati Q uno stato **pozzo** p
 $\overline{Q} = Q \cup \{p\}$.
- **(2)** Per ogni $q \in Q$ e per ogni $a \in \Sigma$,
 $\overline{\delta}(q, a) = \delta(q, a)$ se $\delta(q, a)$ è definita
 $\overline{\delta}(q, a) = p$ altrimenti.
- **(3)** Per ogni $a \in \Sigma$, $\overline{\delta}(p, a) = p$.
- **(4)** $\overline{F} = (Q - F) \cup \{p\}$.

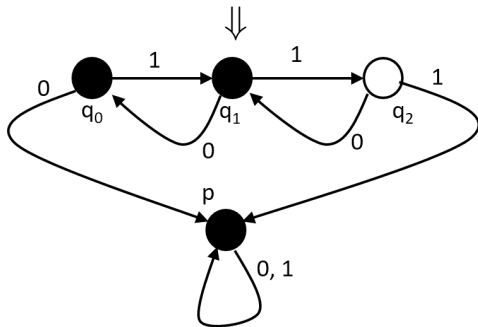
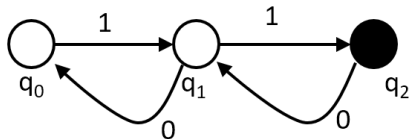
Calcolare l'Automa Complemento di



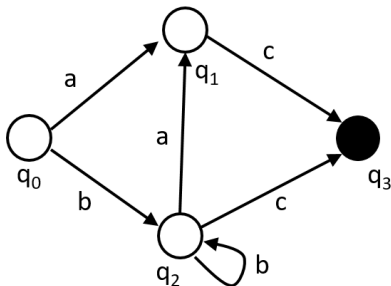
Alfabeto $\Sigma = \{0, 1\}$

Automi a Stati Finiti

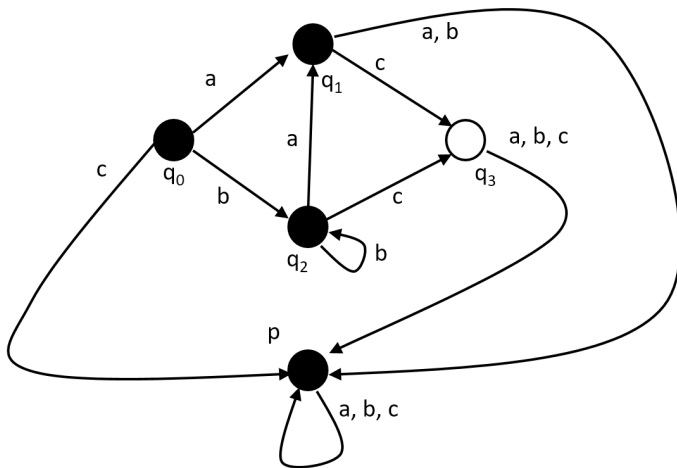
Alfabeto $\Sigma = \{0, 1\}$



Calcolare l'Automa Complemento di



Automa Complemento



Algoritmo per Eliminare le ϵ -mosse

Dato l'automa $M(Q, \Sigma, \delta, q_0, F)$ con ϵ -mosse, vogliamo ottenere l'automa $M'(Q', \Sigma, \delta', q_0, F')$ senza ϵ -mosse

- **Passo 1**

Inserire lo stato q_0 in Q' e in N (insieme dei nuovi stati)

Algoritmo per Eliminare le ϵ -mosse

- **Passo 2**

Impostiamo l'insieme $N' = \emptyset$

Per ogni stato $q \in N$,

copiare da δ tutte le mosse non spontanee che escono da q , cioè $\delta(q, a) = \bar{q}$, creando $\delta'(q, a) = \bar{q}$, aggiungendo $\bar{q}$ in Q' e in N' , se non è già presente in Q' .

Algoritmo per Eliminare le ϵ -mosse

• Passo 3

Per ogni stato $q \in N$,

cercare tutte le transizioni transitive $q \xRightarrow{+} \bar{q}$ che contengono una sola mossa non spontanea preceduta e/o seguita da mosse spontanee,

(cioè $q \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} q_i \xRightarrow{a} \bar{q}$,
 $q \xRightarrow{a} q_i \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} \bar{q}$,
 $q \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} q_i \xRightarrow{a} q_j \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} \bar{q}$).

Creare la corrispondente $\delta'(q, a) = \bar{q}$, aggiungendo $\bar{q}$ in Q' e in N' , se non è già presente in Q' .

ATTENZIONE: gli stati $\bar{q}$ da considerare per i pattern

- $q \xRightarrow{a} q_i \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} \bar{q}$
- $q \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} q_i \xRightarrow{a} q_j \xRightarrow{\epsilon} \dots \xRightarrow{\epsilon} \bar{q}$

sono tutti e solo gli stati $\bar{q}$ dai quali non escono mosse oppure esce almeno una mossa non spontanea (non ci si ferma sugli stati nel mezzo di catene di ϵ dai quali non escono mosse non spontanee, perchè introdurrebbero inutile ridondanza).

Algoritmo per Eliminare le ϵ -mosse

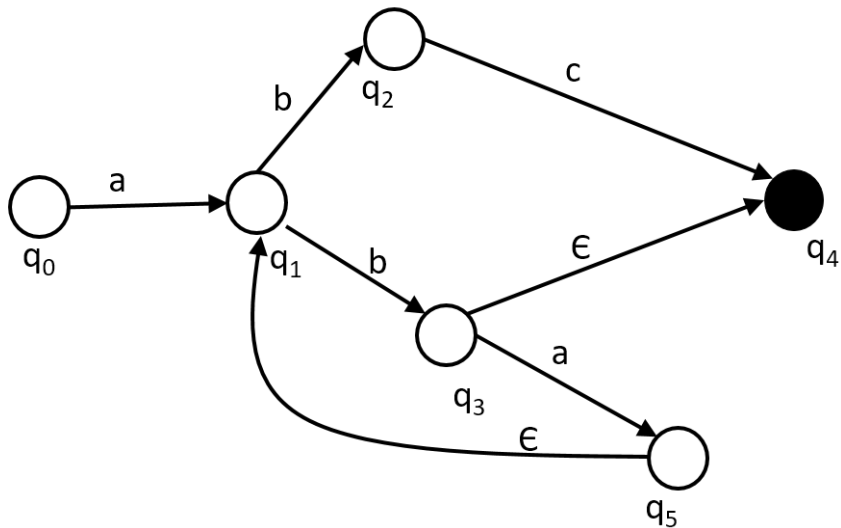
- **Passo 4**

Per ogni stato $q \in N$,
se esiste un percorso che va da q ad uno stato $\bar{q}$
composto solo di ϵ -mosse, con $\bar{q} \in F$,
inserire q in F' (stato finale di M').

- **Passo 5**

Se $N' \neq \emptyset$, $N = N'$ e ripartire dal Passo 2,
altrimenti terminare

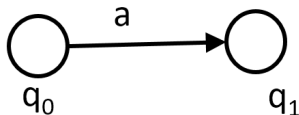
Eliminazione delle ϵ -Mosse: M



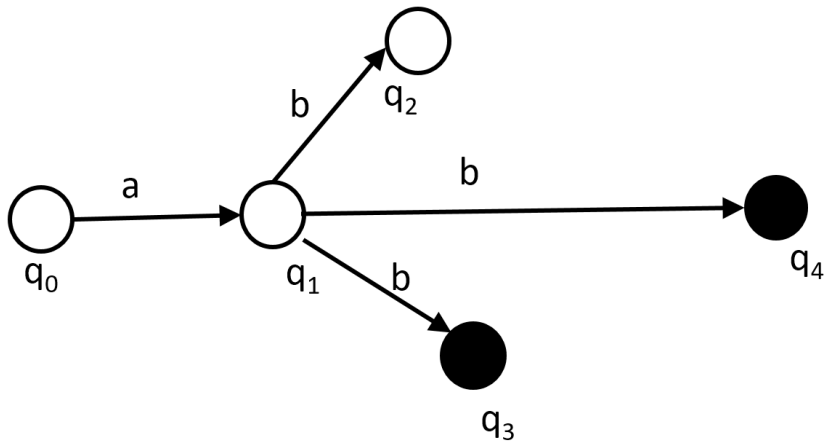
Eliminazione delle ϵ -Mosse: M'



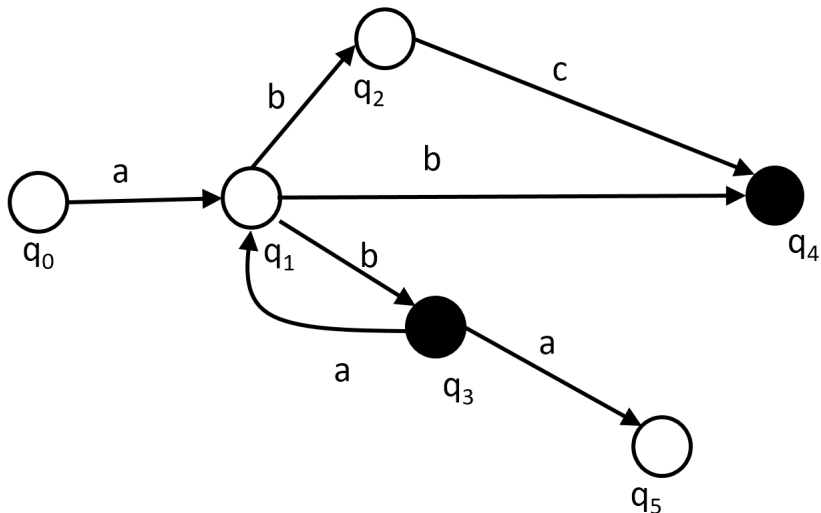
Eliminazione delle ϵ -Mosse: M'



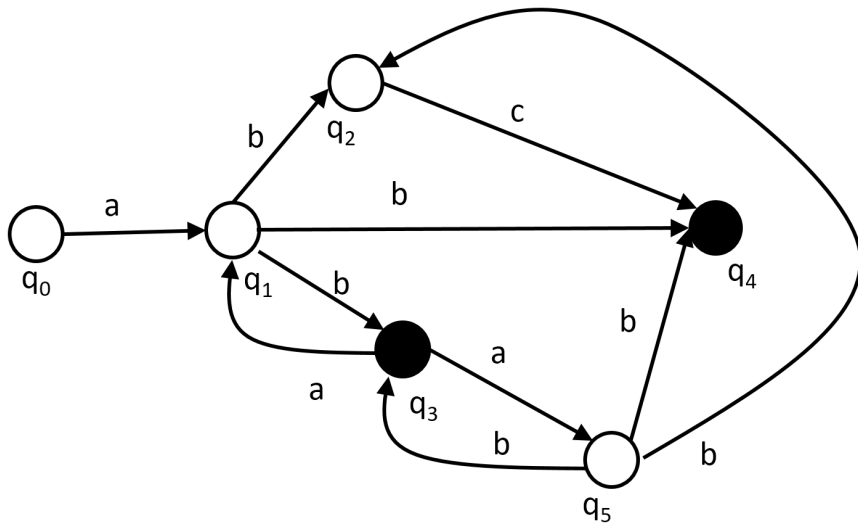
Eliminazione delle ϵ -Mosse: M'



Eliminazione delle ϵ -Mosse: M'



Eliminazione delle ϵ -Mosse: M'



Algoritmo per Eliminare il Non-Determinismo

Dato l'automa $M(Q, \Sigma, \delta, q_o, F)$ Non-Deterministico senza ϵ -mosse, vogliamo ottenere l'automa $M'(Q', \Sigma, \delta', q_0, F')$ Deterministico

- **Passo 1**

Inserire lo stato q_o in Q' e in N (insieme dei nuovi stati)

Algoritmo per Eliminare il Non-Determinismo

• Passo 2

Impostiamo l'insieme $N' = \emptyset$

Per ogni stato $q \in N$ che compare anche in Q ,
per ogni simbolo $a \in \Sigma$ per cui

$\delta(q, a) = \{q_1, q_2, \dots, q_n\}$ non vuoto

- se $n > 1$, creare uno stato **collettivo** $[q_1, q_2, \dots, q_n]$
e aggiungere la mossa $\delta'(q, a) = [q_1, q_2, \dots, q_n]$; se
 $[q_1, q_2, \dots, q_n]$ non già in Q' , inserirlo in Q' e in N' ;
- se $n = 1$, lo stato collettivo diventa uno stato
semplice e lo si aggiunge a Q' e a N' se non è già in
 Q' .

Algoritmo per Eliminare il Non-Determinismo

• Passo 3

Per ogni stato collettivo $[q_1, q_2, \dots, q_n] \in N$,
per ogni stato $q_i \in [q_1, q_2, \dots, q_n]$ e ogni simbolo
 $a \in \Sigma$

calcolare il nuovo stato collettivo

$$[\delta(q_1, a) \cup \delta(q_2, a) \cup \dots \cup \delta(q_n, a)]$$

- aggiungere questo stato in Q' e in N' se non è già in Q' ;

- se il nuovo stato collettivo contiene un solo stato,
diventa uno stato semplice.

- Definire $\delta'([q_1, q_2, \dots, q_n], a) =$
 $[\delta(q_1, a) \cup \delta(q_2, a) \cup \dots \cup \delta(q_n, a)]$

Algoritmo per Eliminare il Non-Determinismo

- **Passo 4**

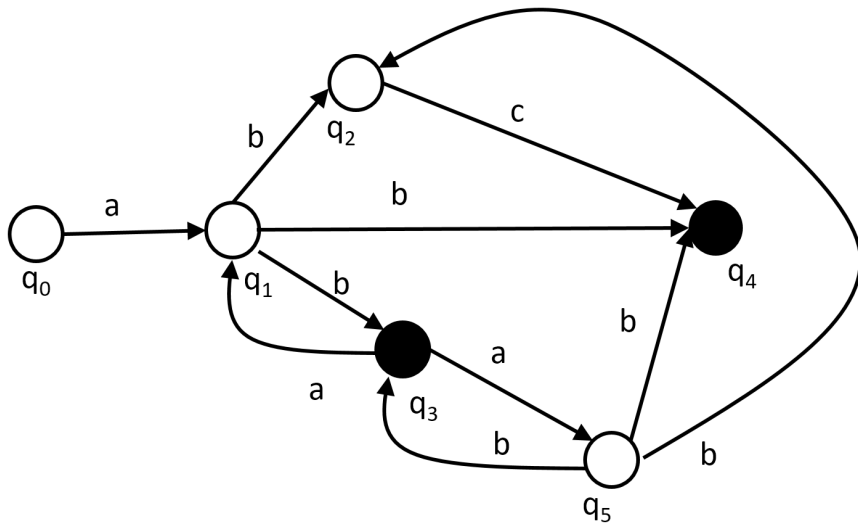
Per ogni stato $q = [q_1, q_2, \dots, q_n] \in N$ (anche con $n = 1$,

se almeno uno stato $q_i \in q$ è finale in M (cioè $q_i \in F$), inserire q negli stati finali di M' , cioè $q \in F'$.

- **Passo 5**

Se $N' \neq \emptyset$, $N = N'$ e ripartire dal Passo 2, altrimenti terminare

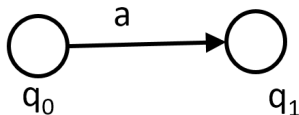
Eliminazione Non-Determinismo: M



Eliminazione Non-Determinismo: M'



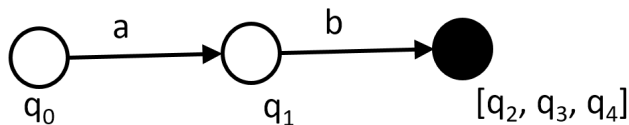
Eliminazione Non-Determinismo: M'



Eliminazione Non-Determinismo: M'

$$\begin{array}{c} q_1 \xRightarrow{a} \emptyset \\ \hline q_1 \xRightarrow{b} \{q_2, q_3, q_4\} \\ \hline q_1 \xRightarrow{c} \emptyset \end{array}$$

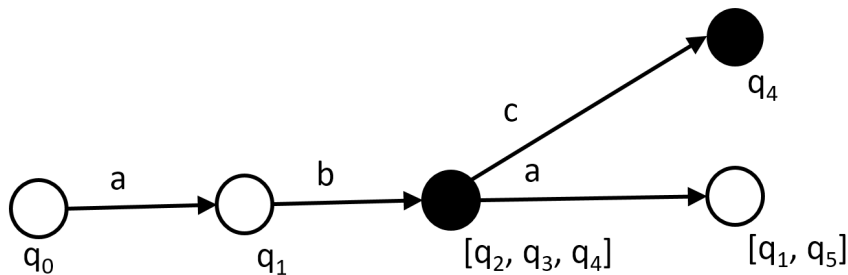
Eliminazione Non-Determinismo: M'



Eliminazione Non-Determinismo: M'

q_2	$\xRightarrow{a}$	$\emptyset$	q_2	$\xRightarrow{b}$	$\emptyset$	q_2	$\xRightarrow{c}$	$\{q_4\}$
q_3	$\xRightarrow{a}$	$\{q_1, q_5\}$	q_3	$\xRightarrow{b}$	$\emptyset$	q_3	$\xRightarrow{c}$	$\emptyset$
q_4	$\xRightarrow{a}$	$\emptyset$	q_4	$\xRightarrow{b}$	$\emptyset$	q_4	$\xRightarrow{c}$	$\emptyset$

Eliminazione Non-Determinismo: M'



Eliminazione Non-Determinismo: M'

$$\begin{array}{lcl} q_1 & \xRightarrow{a} & \emptyset \\ q_5 & \xRightarrow{a} & \emptyset \end{array} \quad \left| \quad \begin{array}{lcl} q_1 & \xRightarrow{b} & \{q_2, q_3, q_4\} \\ q_5 & \xRightarrow{b} & \{q_2, q_3, q_4\} \end{array} \quad \left| \quad \begin{array}{lcl} q_1 & \xRightarrow{c} & \emptyset \\ q_5 & \xRightarrow{c} & \emptyset \end{array}$$

Eliminazione Non-Determinismo: M'

